

SQL PROJECT REPORT

AIRLINE RESERVATION SYSTEM

Submitted By: SIDDHESH THAKARE

Guided By: Mr. Sameer Warsolkar Sir

Course: Data Analyst

Academic Year: 2026-27

1. Abstract

The **Airline Reservation System** is a database management project designed to store and manage information related to airline operations and passenger reservations. The system maintains data about airlines, airports, aircraft, flights, passengers, bookings, seats, and payments in separate but interconnected relational tables.

The database uses **primary keys and foreign keys** to maintain relationships and data integrity between different entities. For example, flights are associated with airlines, aircraft, source airports, and destination airports, while bookings connect passengers with flights. Seat allocation and payment information are separately maintained and linked to bookings.

The project also demonstrates SQL operations such as creating databases and tables, inserting records, filtering records using WHERE, sorting using ORDER BY, and retrieving related information using JOIN. This makes the project useful for understanding the practical implementation of a relational database system.

2. Introduction

An airline reservation system requires the management of a large amount of interconnected information. This includes airline details, airport information, aircraft details, flight schedules, passenger information, reservations, seat assignments, and payment transactions.

The purpose of this project is to implement an **Airline Reservation Database** using SQL. The database is named `airline_reservation`, and it contains separate tables for different components of the reservation process.

The system begins with basic airline and airport information. Each aircraft is associated with an airline, and each flight contains information about its airline, aircraft, source airport, destination airport, departure time, arrival time, fare, and status.

Passenger details are stored independently, while the booking table connects passengers to particular flights. Each booking contains the booking date, travel class, booking status, and total amount. Seat and payment details are then linked to individual bookings.

The project therefore demonstrates how a real-world airline reservation process can be represented using a **relational database model**.

3. Objectives

The main objectives of the Airline Reservation System are:

1. **To create a structured airline database** for storing reservation-related information.
2. **To maintain airline information**, including airline name, country, phone number, and email.
3. **To store airport information**, including airport code, airport name, city, state, and country.
4. **To maintain aircraft information**, including aircraft model, number of seats, manufacturing year, and associated airline.
5. **To manage flight information**, including flight number, airline, aircraft, source, destination, departure time, arrival time, fare, and flight status.
6. **To maintain passenger records**, including name, gender, age, contact information, city, and passport number.
7. **To manage flight bookings** by connecting passengers with flights.
8. **To manage seat allocation** for individual bookings.
9. **To maintain payment information**, including payment method, payment status, payment date, and amount.
10. **To demonstrate SQL querying techniques**, including filtering, sorting, and joining multiple tables.
11. **To maintain data integrity** through primary keys, foreign keys, unique constraints, and NOT NULL constraints.

4. Tools and Technologies

Based on the SQL file, the following tools/technologies are involved:

Tool / Technology	Purpose
SQL	Database creation, table creation, data insertion, and data retrieval
MySQL	The SQL syntax strongly indicates a MySQL-style relational database environment
Relational Database Management System (RDBMS)	Stores data in related tables
Primary Keys	Uniquely identify records
Foreign Keys	Establish relationships between tables
JOIN	Retrieve related information from multiple tables
WHERE	Filter records according to conditions
ORDER BY	Sort query results
DECIMAL	Store monetary values such as fares and payments
DATETIME	Store flight departure and arrival date/time
YEAR	Store aircraft manufacturing year

The file specifically creates the airline_reservation database and uses SQL CREATE TABLE, INSERT INTO, SELECT, WHERE, ORDER BY, and JOIN statements.

Database Design

The database follows a **relational database design**, where different real-world entities are represented as separate tables and relationships are established using foreign keys.

4.1 Table Description

Main Tables

Table	Main Purpose	Primary Key
airline	Stores airline details	AIRLINE_ID
airport	Stores airport details	AIRPORT_ID
aircraft	Stores aircraft information	AIRCRAFT_ID
flight	Stores flight schedules and fares	FLIGHT_ID
passenger	Stores passenger information	PASSENGER_ID
booking	Stores reservation information	BOOKING_ID
seat	Stores seat allocation	SEAT_ID
payment	Stores payment information	PAYMENT_ID

4.2 Table Structure

4.2.1 Airline Table

DESC airline;

Result Grid						
Filter Rows:		Export:		Wrap Cell Content:		
	Field	Type	Null	Key	Default	Extra
▶	AIRLINE_ID	int	NO	PRI	NULL	
	AIRLINE_NAME	varchar(50)	NO		NULL	
	COUNTRY	varchar(40)	NO		NULL	
	PHONE_NO	bigint	YES		NULL	
	EMAIL	varchar(80)	YES		NULL	

4.2.2 Airport Table

DESC airport;

Field	Type	Null	Key	Default	Extra
AIRPORT_ID	int	NO	PRI	NULL	
AIRPORT_CODE	char(3)	NO	UNI	NULL	
AIRPORT_NAME	varchar(80)	NO		NULL	
CITY	varchar(40)	NO		NULL	
STATE	varchar(40)	YES		NULL	
COUNTRY	varchar(40)	NO		NULL	

4.2.3 Aircraft Table

DESC aircraft;

Field	Type	Null	Key	Default	Extra
AIRCRAFT_ID	int	NO	PRI	NULL	
AIRLINE_ID	int	NO	MUL	NULL	
MODEL	varchar(50)	NO		NULL	
TOTAL_SEATS	int	NO		NULL	
MANUFACTURING_YEAR	year	YES		NULL	

4.2.4 Flight Table

DESC flight;

Field	Type	Null	Key	Default	Extra
FLIGHT_ID	int	NO	PRI	NULL	
FLIGHT_NO	varchar(10)	NO	UNI	NULL	
AIRLINE_ID	int	NO	MUL	NULL	
AIRCRAFT_ID	int	NO	MUL	NULL	
SOURCE_AIRPORT_ID	int	NO	MUL	NULL	
DESTINATION_AIRPORT_ID	int	NO	MUL	NULL	
DEPARTURE_TIME	datetime	NO		NULL	
ARRIVAL_TIME	datetime	NO		NULL	
BASE_FARE	decimal(10,2)	NO		NULL	
STATUS	varchar(20)	YES		Scheduled	

4.2.5 Passenger Table

DESC passenger;

Result Grid


Filter Rows:

Export:


Wrap Cell Content:


	Field	Type	Null	Key	Default	Extra
▶	PASSENGER_ID	int	NO	PRI	NULL	
	FIRST_NAME	varchar(40)	NO		NULL	
	LAST_NAME	varchar(40)	NO		NULL	
	GENDER	varchar(10)	YES		NULL	
	AGE	int	YES		NULL	
	PHONE_NO	bigint	NO		NULL	
	EMAIL	varchar(80)	YES		NULL	
	CITY	varchar(40)	YES		NULL	
	PASSPORT_NO	varchar(20)	YES	UNI	NULL	

4.2.6 Booking Table

DESC booking;

Result Grid


Filter Rows:

Export:


Wrap Cell Content:


	Field	Type	Null	Key	Default	Extra
▶	BOOKING_ID	int	NO	PRI	NULL	
	PASSENGER_ID	int	NO	MUL	NULL	
	FLIGHT_ID	int	NO	MUL	NULL	
	BOOKING_DATE	date	NO		NULL	
	CLASS	varchar(20)	NO		NULL	
	BOOKING_STATUS	varchar(20)	YES		Confirmed	
	TOTAL_AMOUNT	decimal(10,2)	NO		NULL	

4.2.7 Seats Table

DESC table;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

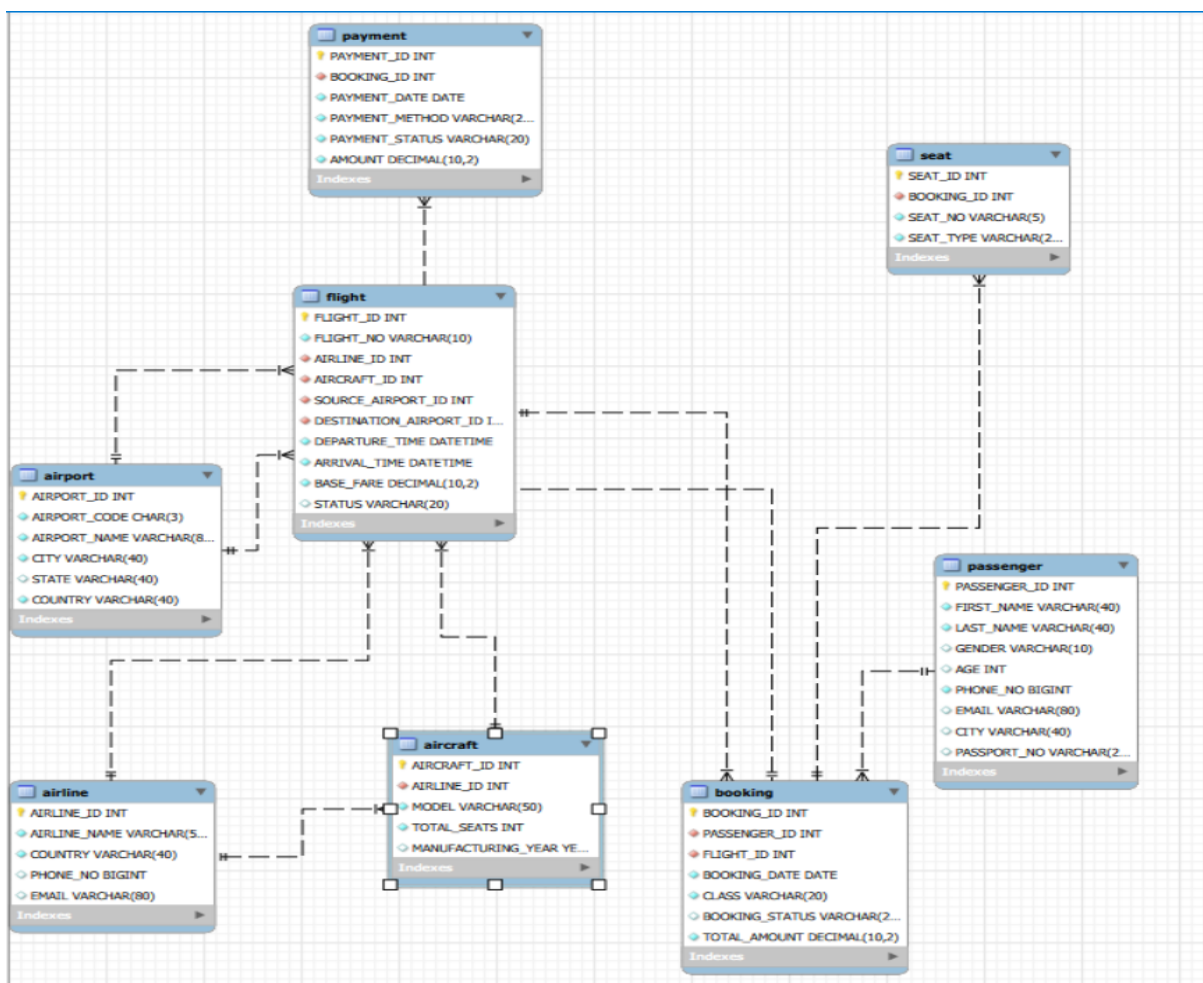
	Field	Type	Null	Key	Default	Extra
▶	SEAT_ID	int	NO	PRI	NULL	
	BOOKING_ID	int	NO	UNI	NULL	
	SEAT_NO	varchar(5)	NO		NULL	
	SEAT_TYPE	varchar(20)	NO		NULL	

4.2.8 Payment Table

DESC payment;

Field	Type	Null	Key	Default	Extra
PAYMENT_ID	int	NO	PRI	NULL	
BOOKING_ID	int	NO	UNI	NULL	
PAYMENT_DATE	date	NO		NULL	
PAYMENT_METHOD	varchar(20)	NO		NULL	
PAYMENT_STATUS	varchar(20)	NO		NULL	
AMOUNT	decimal(10,2)	NO		NULL	

4.3 Relationship Diagram of Airline Reservation System



SQL Queries and Analysis

Database Creation:

```
CREATE DATABASE airline_reservation;
```

```
USE airline_reservation;
```

Table creation:

1. Airline

```
CREATE TABLE airline (  
    AIRLINE_ID INT PRIMARY KEY,  
    AIRLINE_NAME VARCHAR(50) NOT NULL,  
    COUNTRY VARCHAR(40) NOT NULL,  
    PHONE_NO BIGINT,  
    EMAIL VARCHAR(80)  
);
```

2. Airport

```
CREATE TABLE airport (  
    AIRPORT_ID INT PRIMARY KEY,  
    AIRPORT_CODE CHAR(3) UNIQUE NOT NULL,  
    AIRPORT_NAME VARCHAR(80) NOT NULL,  
    CITY VARCHAR(40) NOT NULL,  
    STATE VARCHAR(40),  
    COUNTRY VARCHAR(40) NOT NULL  
);
```

3. Aircraft

```
CREATE TABLE aircraft (  
    AIRCRAFT_ID INT PRIMARY KEY,  
    AIRLINE_ID INT NOT NULL,  
    MODEL VARCHAR(50) NOT NULL,  
    TOTAL_SEATS INT NOT NULL,  
    MANUFACTURING_YEAR YEAR,  
    FOREIGN KEY (AIRLINE_ID)  
        REFERENCES airline(AIRLINE_ID)  
);
```

4. Flight

```
CREATE TABLE flight (  
    FLIGHT_ID INT PRIMARY KEY,  
    FLIGHT_NO VARCHAR(10) UNIQUE NOT NULL,  
    AIRLINE_ID INT NOT NULL,  
    AIRCRAFT_ID INT NOT NULL,  
    SOURCE_AIRPORT_ID INT NOT NULL,  
    DESTINATION_AIRPORT_ID INT NOT NULL,  
    DEPARTURE_TIME DATETIME NOT NULL,  
    ARRIVAL_TIME DATETIME NOT NULL,  
    BASE_FARE DECIMAL(10,2) NOT NULL,  
    STATUS VARCHAR(20) DEFAULT 'Scheduled',  
  
    FOREIGN KEY (AIRLINE_ID)  
        REFERENCES airline(AIRLINE_ID),  
  
    FOREIGN KEY (AIRCRAFT_ID)  
        REFERENCES aircraft(AIRCRAFT_ID),  
  
    FOREIGN KEY (SOURCE_AIRPORT_ID)  
        REFERENCES airport(AIRPORT_ID),  
  
    FOREIGN KEY (DESTINATION_AIRPORT_ID)  
        REFERENCES airport(AIRPORT_ID)  
);
```

5. Passenger

```
CREATE TABLE passenger (  
    PASSENGER_ID INT PRIMARY KEY,  
    FIRST_NAME VARCHAR(40) NOT NULL,  
    LAST_NAME VARCHAR(40) NOT NULL,  
    GENDER VARCHAR(10),  
    AGE INT,  
    PHONE_NO BIGINT NOT NULL,  
    EMAIL VARCHAR(80),  
    CITY VARCHAR(40),  
    PASSPORT_NO VARCHAR(20) UNIQUE  
);
```

6. Booking

```
CREATE TABLE booking (  
    BOOKING_ID INT PRIMARY KEY,  
    PASSENGER_ID INT NOT NULL,  
    FLIGHT_ID INT NOT NULL,  
    BOOKING_DATE DATE NOT NULL,  
    CLASS VARCHAR(20) NOT NULL,  
    BOOKING_STATUS VARCHAR(20) DEFAULT 'Confirmed',  
    TOTAL_AMOUNT DECIMAL(10,2) NOT NULL,  
  
    FOREIGN KEY (PASSENGER_ID)  
        REFERENCES passenger(PASSENGER_ID),  
  
    FOREIGN KEY (FLIGHT_ID)  
        REFERENCES flight(FLIGHT_ID)  
);
```

7. Seats

```
CREATE TABLE seat (  
    SEAT_ID INT PRIMARY KEY,  
    BOOKING_ID INT UNIQUE NOT NULL,  
    SEAT_NO VARCHAR(5) NOT NULL,  
    SEAT_TYPE VARCHAR(20) NOT NULL,  
  
    FOREIGN KEY (BOOKING_ID)  
        REFERENCES booking(BOOKING_ID)  
);
```

8. Payment

```
CREATE TABLE payment (  
    PAYMENT_ID INT PRIMARY KEY,  
    BOOKING_ID INT UNIQUE NOT NULL,  
    PAYMENT_DATE DATE NOT NULL,  
    PAYMENT_METHOD VARCHAR(20) NOT NULL,  
    PAYMENT_STATUS VARCHAR(20) NOT NULL,  
    AMOUNT DECIMAL(10,2) NOT NULL,  
  
    FOREIGN KEY (BOOKING_ID)  
        REFERENCES booking(BOOKING_ID)  
);
```

Data Insertion

1. Insert Into Airline Table

INSERT INTO airline

(AIRLINE_ID, AIRLINE_NAME, COUNTRY, PHONE_NO, EMAIL)

VALUES

(1, 'Air India', 'India', 18002331401, 'support@airindia.com'),

(2, 'IndiGo', 'India', 18001803838, 'support@indigo.com'),

(3, 'Vistara', 'India', 9289228888, 'support@vistara.com'),

(4, 'Emirates', 'UAE', 800777777, 'support@emirates.com'),

(5, 'Singapore Airlines', 'Singapore', 6222118888, 'support@sia.com');

SELECT * from airline;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	AIRLINE_ID	AIRLINE_NAME	COUNTRY	PHONE_NO	EMAIL
▶	1	Air India	India	18002331401	support@airindia.com
	2	IndiGo	India	18001803838	support@indigo.com
	3	Vistara	India	9289228888	support@vistara.com
	4	Emirates	UAE	800777777	support@emirates.com
	5	Singapore Airlines	Singapore	6222118888	support@sia.com
✱	NULL	NULL	NULL	NULL	NULL

2. Insert Into Airport Table

INSERT INTO airport

(AIRPORT_ID, AIRPORT_CODE, AIRPORT_NAME, CITY, STATE, COUNTRY)

VALUES

(101, 'BOM', 'Chhatrapati Shivaji Maharaj International Airport', 'Mumbai', 'Maharashtra', 'India'),

(102, 'DEL', 'Indira Gandhi International Airport', 'Delhi', 'Delhi', 'India'),

(103, 'BLR', 'Kempegowda International Airport', 'Bengaluru', 'Karnataka', 'India'),

(104, 'MAA', 'Chennai International Airport', 'Chennai', 'Tamil Nadu', 'India'),

(105, 'HYD', 'Rajiv Gandhi International Airport', 'Hyderabad', 'Telangana', 'India'),

(106, 'CCU', 'Netaji Subhas Chandra Bose International Airport', 'Kolkata', 'West Bengal', 'India'),

(107, 'PNQ', 'Pune International Airport', 'Pune', 'Maharashtra', 'India'),

(108, 'GOI', 'Goa International Airport', 'Goa', 'Goa', 'India'),

(109, 'DXB', 'Dubai International Airport', 'Dubai', 'Dubai', 'UAE'),

(110, 'SIN', 'Singapore Changi Airport', 'Singapore', 'Singapore', 'Singapore');

SELECT * from airport;

Result Grid						
Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:
AIRPORT_ID	AIRPORT_CODE	AIRPORT_NAME	CITY	STATE	COUNTRY	
101	BOM	Chhatrapati Shivaji Maharaj International Airport	Mumbai	Maharashtra	India	
102	DEL	Indira Gandhi International Airport	Delhi	Delhi	India	
103	BLR	Kempegowda International Airport	Bengaluru	Karnataka	India	
104	MAA	Chennai International Airport	Chennai	Tamil Nadu	India	
105	HYD	Rajiv Gandhi International Airport	Hyderabad	Telangana	India	
106	CCU	Netaji Subhas Chandra Bose International Airport	Kolkata	West Bengal	India	
107	PNQ	Pune International Airport	Pune	Maharashtra	India	
108	GOI	Goa International Airport	Goa	Goa	India	
109	DXB	Dubai International Airport	Dubai	Dubai	UAE	
110	SIN	Singapore Changi Airport	Singapore	Singapore	Singapore	
	NULL	NULL	NULL	NULL	NULL	

3. Insert Into Aircraft Table

INSERT INTO aircraft

(AIRCRAFT_ID, AIRLINE_ID, MODEL, TOTAL_SEATS, MANUFACTURING_YEAR)

VALUES

(201, 1, 'Airbus A320', 180, 2019),

(202, 1, 'Boeing 787', 256, 2020),

(203, 2, 'Airbus A320neo', 186, 2021),

(204, 2, 'Airbus A321neo', 222, 2022),

(205, 3, 'Airbus A321', 188, 2018),

(206, 4, 'Boeing 777-300ER', 354, 2019),

(207, 4, 'Airbus A380', 484, 2020),

(208, 5, 'Airbus A350', 253, 2021),

(209, 5, 'Boeing 787-10', 337, 2022),

(210, 3, 'Boeing 737', 160, 2017);

SELECT * from aircraft;

Result Grid					
Filter Rows:		Edit:    Export/Import:   Wrap Cell Content: 			
	AIRCRAFT_ID	AIRLINE_ID	MODEL	TOTAL_SEATS	MANUFACTURING_YEAR
▶	201	1	Airbus A320	180	2019
	202	1	Boeing 787	256	2020
	203	2	Airbus A320neo	186	2021
	204	2	Airbus A321neo	222	2022
	205	3	Airbus A321	188	2018
	206	4	Boeing 777-300ER	354	2019
	207	4	Airbus A380	484	2020
	208	5	Airbus A350	253	2021
	209	5	Boeing 787-10	337	2022
	210	3	Boeing 737	160	2017
*	NULL	NULL	NULL	NULL	NULL

4. Insert Into Flight Table

INSERT INTO flight

(FLIGHT_ID, FLIGHT_NO, AIRLINE_ID, AIRCRAFT_ID,
SOURCE_AIRPORT_ID, DESTINATION_AIRPORT_ID,
DEPARTURE_TIME, ARRIVAL_TIME, BASE_FARE, STATUS)

VALUES

(301, 'AI101', 1, 201, 101, 102,
'2026-09-01 06:30:00', '2026-09-01 08:45:00', 5500.00, 'Scheduled'),

(302, 'AI102', 1, 202, 102, 101,
'2026-09-02 10:00:00', '2026-09-02 12:20:00', 6200.00, 'Scheduled'),

(303, '6E201', 2, 203, 101, 103,
'2026-09-03 07:15:00', '2026-09-03 09:05:00', 4800.00, 'Scheduled'),

(304, '6E202', 2, 204, 103, 101,
'2026-09-04 18:00:00', '2026-09-04 20:00:00', 5100.00, 'Scheduled'),

(305, 'UK301', 3, 205, 101, 104,
'2026-09-05 09:30:00', '2026-09-05 11:40:00', 6700.00, 'Scheduled'),

(306, 'UK302', 3, 210, 104, 101,
'2026-09-06 15:00:00', '2026-09-06 17:15:00', 6900.00, 'Scheduled'),

(307, 'AI205', 1, 201, 101, 105,
'2026-09-07 11:00:00', '2026-09-07 12:50:00', 4300.00, 'Scheduled'),

(308, '6E305', 2, 203, 102, 106,
'2026-09-08 13:00:00', '2026-09-08 15:20:00', 5900.00, 'Scheduled'),

(309, 'EK501', 4, 206, 101, 109,
'2026-09-09 04:00:00', '2026-09-09 06:15:00', 18500.00, 'Scheduled'),

(310, 'EK502', 4, 207, 109, 101,
'2026-09-10 14:00:00', '2026-09-10 19:00:00', 20500.00, 'Scheduled'),

(311, 'SQ421', 5, 208, 101, 110,
'2026-09-11 22:00:00', '2026-09-12 06:00:00', 22000.00, 'Scheduled'),

(312, 'SQ422', 5, 209, 110, 101,
'2026-09-13 09:00:00', '2026-09-13 17:00:00', 22500.00, 'Scheduled'),

(313, 'AI410', 1, 202, 101, 107,
'2026-09-14 08:00:00', '2026-09-14 09:10:00', 3200.00, 'Scheduled'),

```
(314, '6E411', 2, 204, 107, 101,
'2026-09-15 19:00:00', '2026-09-15 20:10:00', 3500.00, 'Scheduled'),

(315, 'AI510', 1, 201, 101, 108,
'2026-09-16 16:00:00', '2026-09-16 17:15:00', 4200.00, 'Scheduled');
```

SELECT * from flight;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	FLIGHT_ID	FLIGHT_NO	AIRLINE_ID	AIRCRAFT_ID	SOURCE_AIRPORT_ID	DESTINATION_AIRPORT_ID	DEPARTURE_TIME	ARRIVAL_TIME	BASE_FARE	STATUS
▶	301	AI101	1	201	101	102	2026-09-01 06:30:00	2026-09-01 08:45:00	5500.00	Scheduled
	302	AI102	1	202	102	101	2026-09-02 10:00:00	2026-09-02 12:20:00	6200.00	Scheduled
	303	6E201	2	203	101	103	2026-09-03 07:15:00	2026-09-03 09:05:00	4800.00	Scheduled
	304	6E202	2	204	103	101	2026-09-04 18:00:00	2026-09-04 20:00:00	5100.00	Scheduled
	305	UK301	3	205	101	104	2026-09-05 09:30:00	2026-09-05 11:40:00	6700.00	Scheduled
	306	UK302	3	210	104	101	2026-09-06 15:00:00	2026-09-06 17:15:00	6900.00	Scheduled
	307	AI205	1	201	101	105	2026-09-07 11:00:00	2026-09-07 12:50:00	4300.00	Scheduled
	308	6E305	2	203	102	106	2026-09-08 13:00:00	2026-09-08 15:20:00	5900.00	Scheduled
	309	EK501	4	206	101	109	2026-09-09 04:00:00	2026-09-09 06:15:00	18500.00	Scheduled
	310	EK502	4	207	109	101	2026-09-10 14:00:00	2026-09-10 19:00:00	20500.00	Scheduled
	311	SQ421	5	208	101	110	2026-09-11 22:00:00	2026-09-12 06:00:00	22000.00	Scheduled
	312	SQ422	5	209	110	101	2026-09-13 09:00:00	2026-09-13 17:00:00	22500.00	Scheduled
	313	AI410	1	202	101	107	2026-09-14 08:00:00	2026-09-14 09:10:00	3200.00	Scheduled
	314	6E411	2	204	107	101	2026-09-15 19:00:00	2026-09-15 20:10:00	3500.00	Scheduled
	315	AI510	1	201	101	108	2026-09-16 16:00:00	2026-09-16 17:15:00	4200.00	Scheduled
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5. Insert Into Passenger Table

INSERT INTO passenger

(PASSENGER_ID, FIRST_NAME, LAST_NAME, GENDER, AGE, PHONE_NO, EMAIL, CITY, PASSPORT_NO)

VALUES

(1, 'Rahul', 'Sharma', 'Male', 28, 9876500001, 'rahul.sharma@gmail.com', 'Mumbai', 'P100001'),

(2, 'Priya', 'Patel', 'Female', 31, 9876500002, 'priya.patel@gmail.com', 'Pune', 'P100002'),

(3, 'Amit', 'Verma', 'Male', 35, 9876500003, 'amit.verma@gmail.com', 'Delhi', 'P100003'),

(4, 'Sneha', 'Desai', 'Female', 26, 9876500004, 'sneha.desai@gmail.com', 'Mumbai', 'P100004'),

(5, 'Rohan', 'Mehta', 'Male', 42, 9876500005, 'rohan.mehta@gmail.com', 'Ahmedabad', 'P100005'),

(6, 'Anjali', 'Kumar', 'Female', 29, 9876500006, 'anjali.kumar@gmail.com', 'Delhi', 'P100006'),

(7, 'Vikram', 'Singh', 'Male', 38, 9876500007, 'vikram.singh@gmail.com', 'Jaipur', 'P100007'),

(8, 'Neha', 'Joshi', 'Female', 24, 9876500008, 'neha.joshi@gmail.com', 'Pune', 'P100008'),

(9, 'Arjun', 'Nair', 'Male', 33, 9876500009, 'arjun.nair@gmail.com', 'Kochi', 'P100009'),

(10, 'Kavita', 'Shah', 'Female', 40, 9876500010, 'kavita.shah@gmail.com', 'Mumbai', 'P100010'),

(11, 'Sanjay', 'Rao', 'Male', 45, 9876500011, 'sanjay.rao@gmail.com', 'Bengaluru', 'P100011'),

(12, 'Pooja', 'Iyer', 'Female', 27, 9876500012, 'pooja.iyer@gmail.com', 'Chennai', 'P100012'),

(13, 'Karan', 'Malhotra', 'Male', 30, 9876500013, 'karan.malhotra@gmail.com', 'Delhi', 'P100013'),

(14, 'Riya', 'Kapoor', 'Female', 22, 9876500014, 'riya.kapoor@gmail.com', 'Mumbai', 'P100014'),

(15, 'Nikhil', 'Gupta', 'Male', 36, 9876500015, 'nikhil.gupta@gmail.com', 'Pune', 'P100015'),

(16, 'Meera', 'Chopra', 'Female', 34, 9876500016, 'meera.chopra@gmail.com', 'Kolkata', 'P100016'),

(17, 'Aditya', 'Kulkarni', 'Male', 25, 9876500017, 'aditya.kulkarni@gmail.com', 'Pune', 'P100017'),

(18, 'Simran', 'Kaur', 'Female', 29, 9876500018, 'simran.kaur@gmail.com', 'Amritsar', 'P100018'),

(19, 'Manish', 'Yadav', 'Male', 41, 9876500019, 'manish.yadav@gmail.com', 'Lucknow', 'P100019'),

(20, 'Ayesha', 'Khan', 'Female', 32, 9876500020, 'ayesha.khan@gmail.com', 'Mumbai', 'P100020'),

(21, 'Varun', 'Bose', 'Male', 37, 9876500021, 'varun.bose@gmail.com', 'Kolkata', 'P100021'),

(22, 'Tanya', 'Sen', 'Female', 26, 9876500022, 'tanya.sen@gmail.com', 'Kolkata', 'P100022'),

(23, 'Deepak', 'Mishra', 'Male', 48, 9876500023, 'deepak.mishra@gmail.com', 'Varanasi', 'P100023'),

(24, 'Isha', 'Reddy', 'Female', 23, 9876500024, 'isha.reddy@gmail.com', 'Hyderabad', 'P100024'),

(25, 'Mohit', 'Agarwal', 'Male', 39, 9876500025, 'mohit.agarwal@gmail.com', 'Jaipur', 'P100025'),

(26, 'Divya', 'Menon', 'Female', 30, 9876500026, 'divya.menon@gmail.com', 'Kochi', 'P100026'),

(27, 'Suresh', 'Pillai', 'Male', 52, 9876500027, 'suresh.pillai@gmail.com', 'Kochi', 'P100027'),

(28, 'Nandini', 'Rao', 'Female', 28, 9876500028, 'nandini.rao@gmail.com', 'Bengaluru', 'P100028'),

(29, 'Akash', 'Patil', 'Male', 27, 9876500029, 'akash.patil@gmail.com', 'Mumbai', 'P100029'),

(30, 'Shreya', 'Das', 'Female', 33, 9876500030, 'shreya.das@gmail.com', 'Kolkata', 'P100030'),

(31, 'Harsh', 'Thakur', 'Male', 31, 9876500031, 'harsh.thakur@gmail.com', 'Delhi', 'P100031'),

(32, 'Mansi', 'Joshi', 'Female', 25, 9876500032, 'mansi.joshi@gmail.com', 'Pune', 'P100032'),

(33, 'Raj', 'Chavan', 'Male', 44, 9876500033, 'raj.chavan@gmail.com', 'Mumbai', 'P100033'),

(34, 'Swati', 'Naik', 'Female', 36, 9876500034, 'swati.naik@gmail.com', 'Goa', 'P100034'),

(35, 'Yash', 'Saxena', 'Male', 29, 9876500035, 'yash.saxena@gmail.com', 'Delhi', 'P100035'),

(36, 'Komal', 'Bhat', 'Female', 27, 9876500036, 'komal.bhat@gmail.com', 'Bengaluru', 'P100036'),

(37, 'Abhishek', 'Jain', 'Male', 40, 9876500037, 'abhishek.jain@gmail.com', 'Indore', 'P100037'),

6. Insert Into Booking Table

INSERT INTO booking

(BOOKING_ID, PASSENGER_ID, FLIGHT_ID, BOOKING_DATE, CLASS,
BOOKING_STATUS, TOTAL_AMOUNT)

VALUES

(5001, 1, 301, '2026-08-01', 'Economy', 'Confirmed', 5500.00),

(5002, 2, 301, '2026-08-02', 'Business', 'Confirmed', 9500.00),

(5003, 3, 302, '2026-08-03', 'Economy', 'Confirmed', 6200.00),

(5004, 4, 303, '2026-08-04', 'Economy', 'Confirmed', 4800.00),

(5005, 5, 304, '2026-08-04', 'Business', 'Confirmed', 8500.00),

(5006, 6, 305, '2026-08-05', 'Economy', 'Confirmed', 6700.00),

(5007, 7, 306, '2026-08-05', 'Economy', 'Cancelled', 0.00),

(5008, 8, 307, '2026-08-05', 'Economy', 'Confirmed', 4300.00),

(5009, 9, 308, '2026-08-05', 'Business', 'Confirmed', 9000.00),

(5010, 10, 309, '2026-08-06', 'Economy', 'Confirmed', 18500.00),

(5011, 11, 310, '2026-08-06', 'Business', 'Confirmed', 32000.00),

(5012, 12, 311, '2026-08-06', 'Economy', 'Confirmed', 22000.00),

(5013, 13, 312, '2026-08-07', 'Economy', 'Confirmed', 22500.00),

(5014, 14, 313, '2026-08-07', 'Economy', 'Confirmed', 3200.00),

(5015, 15, 314, '2026-08-07', 'Business', 'Confirmed', 6500.00),

(5016, 16, 315, '2026-08-08', 'Economy', 'Confirmed', 4200.00),

(5017, 17, 301, '2026-08-08', 'Economy', 'Confirmed', 5500.00),

(5018, 18, 302, '2026-08-08', 'Economy', 'Confirmed', 6200.00),

(5019, 19, 303, '2026-08-09', 'Business', 'Confirmed', 8000.00),

(5020, 20, 304, '2026-08-09', 'Economy', 'Confirmed', 5100.00),

(5021, 21, 305, '2026-08-09', 'Economy', 'Confirmed', 6700.00),

(5022, 22, 306, '2026-08-10', 'Business', 'Confirmed', 10500.00),
(5023, 23, 307, '2026-08-10', 'Economy', 'Confirmed', 4300.00),
(5024, 24, 308, '2026-08-10', 'Economy', 'Confirmed', 5900.00),
(5025, 25, 309, '2026-08-11', 'Business', 'Confirmed', 28000.00),
(5026, 26, 310, '2026-08-11', 'Economy', 'Confirmed', 20500.00),
(5027, 27, 311, '2026-08-11', 'Business', 'Confirmed', 35000.00),
(5028, 28, 312, '2026-08-12', 'Economy', 'Confirmed', 22500.00),
(5029, 29, 313, '2026-08-12', 'Economy', 'Confirmed', 3200.00),
(5030, 30, 314, '2026-08-12', 'Business', 'Confirmed', 6500.00),

(5031, 31, 315, '2026-08-13', 'Economy', 'Confirmed', 4200.00),
(5032, 32, 301, '2026-08-13', 'Economy', 'Confirmed', 5500.00),
(5033, 33, 302, '2026-08-13', 'Business', 'Confirmed', 9500.00),
(5034, 34, 303, '2026-08-14', 'Economy', 'Confirmed', 4800.00),
(5035, 35, 304, '2026-08-14', 'Economy', 'Confirmed', 5100.00),
(5036, 36, 305, '2026-08-14', 'Business', 'Confirmed', 11500.00),
(5037, 37, 306, '2026-08-15', 'Economy', 'Confirmed', 6900.00),
(5038, 38, 307, '2026-08-15', 'Economy', 'Confirmed', 4300.00),
(5039, 39, 308, '2026-08-15', 'Business', 'Confirmed', 9000.00),
(5040, 40, 309, '2026-08-16', 'Economy', 'Confirmed', 18500.00);

SELECT * from booking;

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
	BOOKING_ID	PASSENGER_ID	FLIGHT_ID	BOOKING_DATE	CLASS	BOOKING_STATUS	TOTAL_AMOUNT		
	5001	1	301	2026-08-01	Economy	Confirmed	5500.00		
	5002	2	301	2026-08-02	Business	Confirmed	9500.00		
	5003	3	302	2026-08-03	Economy	Confirmed	6200.00		
	5004	4	303	2026-08-04	Economy	Confirmed	4800.00		
	5005	5	304	2026-08-04	Business	Confirmed	8500.00		
	5006	6	305	2026-08-05	Economy	Confirmed	6700.00		
	5007	7	306	2026-08-05	Economy	Cancelled	0.00		
	5008	8	307	2026-08-05	Economy	Confirmed	4300.00		
	5009	9	308	2026-08-05	Business	Confirmed	9000.00		
	5010	10	309	2026-08-06	Economy	Confirmed	18500.00		
	5011	11	310	2026-08-06	Business	Confirmed	32000.00		
	5012	12	311	2026-08-06	Economy	Confirmed	22000.00		
	5013	13	312	2026-08-07	Economy	Confirmed	22500.00		
	5014	14	313	2026-08-07	Economy	Confirmed	3200.00		
	5015	15	314	2026-08-07	Business	Confirmed	6500.00		
	5016	16	315	2026-08-08	Economy	Confirmed	4200.00		
	5017	17	301	2026-08-08	Economy	Confirmed	5500.00		
	5018	18	302	2026-08-08	Economy	Confirmed	6200.00		
	5019	19	303	2026-08-09	Business	Confirmed	8000.00		
Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
	BOOKING_ID	PASSENGER_ID	FLIGHT_ID	BOOKING_DATE	CLASS	BOOKING_STATUS	TOTAL_AMOUNT		
	5020	20	304	2026-08-09	Economy	Confirmed	5100.00		
	5021	21	305	2026-08-09	Economy	Confirmed	6700.00		
	5022	22	306	2026-08-10	Business	Confirmed	10500.00		
	5023	23	307	2026-08-10	Economy	Confirmed	4300.00		
	5024	24	308	2026-08-10	Economy	Confirmed	5900.00		
	5025	25	309	2026-08-11	Business	Confirmed	28000.00		
	5026	26	310	2026-08-11	Economy	Confirmed	20500.00		
	5027	27	311	2026-08-11	Business	Confirmed	35000.00		
	5028	28	312	2026-08-12	Economy	Confirmed	22500.00		
	5029	29	313	2026-08-12	Economy	Confirmed	3200.00		
	5030	30	314	2026-08-12	Business	Confirmed	6500.00		
	5031	31	315	2026-08-13	Economy	Confirmed	4200.00		
	5032	32	301	2026-08-13	Economy	Confirmed	5500.00		
	5033	33	302	2026-08-13	Business	Confirmed	9500.00		
	5034	34	303	2026-08-14	Economy	Confirmed	4800.00		
	5035	35	304	2026-08-14	Economy	Confirmed	5100.00		
	5036	36	305	2026-08-14	Business	Confirmed	11500.00		
	5037	37	306	2026-08-15	Economy	Confirmed	6900.00		
	5038	38	307	2026-08-15	Economy	Confirmed	4300.00		
	5039	39	308	2026-08-15	Busine Economy	Confirmed	9000.00		
	5040	40	309	2026-08-16	Economy	Confirmed	18500.00		
	NULL	NULL	NULL	NULL	NULL	NULL	NULL		

7. Insert Into Seat Table

INSERT INTO seat

(SEAT_ID, BOOKING_ID, SEAT_NO, SEAT_TYPE)

VALUES

(6001,5001,'1A','Window'),

(6002,5002,'1B','Window'),

(6003,5003,'2A','Window'),

(6004,5004,'2B','Middle'),

(6005,5005,'3A','Window'),

(6006,5006,'3B','Middle'),

(6007,5007,'4A','Window'),

(6008,5008,'4B','Middle'),

(6009,5009,'5A','Window'),

(6010,5010,'5B','Middle'),

(6011,5011,'6A','Window'),

(6012,5012,'6B','Middle'),

(6013,5013,'7A','Window'),

(6014,5014,'7B','Middle'),

(6015,5015,'8A','Window'),

(6016,5016,'8B','Middle'),

(6017,5017,'9A','Window'),

(6018,5018,'9B','Middle'),

(6019,5019,'10A','Window'),

(6020,5020,'10B','Middle'),

(6021,5021,'11A','Window'),

(6022,5022,'11B','Middle'),

(6023,5023,'12A','Window'),

(6024,5024,'12B','Middle'),

(6025,5025,'13A','Window'),
(6026,5026,'13B','Middle'),
(6027,5027,'14A','Window'),
(6028,5028,'14B','Middle'),
(6029,5029,'15A','Window'),
(6030,5030,'15B','Middle'),
(6031,5031,'16A','Window'),
(6032,5032,'16B','Middle'),
(6033,5033,'17A','Window'),
(6034,5034,'17B','Middle'),
(6035,5035,'18A','Window'),
(6036,5036,'18B','Middle'),
(6037,5037,'19A','Window'),
(6038,5038,'19B','Middle'),
(6039,5039,'20A','Window'),
(6040,5040,'20B','Middle');

SELECT * from seat;

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
SEAT_ID	BOOKING_ID	SEAT_NO	SEAT_TYPE	
6001	5001	1A	Window	
6002	5002	1B	Window	
6003	5003	2A	Window	
6004	5004	2B	Middle	
6005	5005	3A	Window	
6006	5006	3B	Middle	
6007	5007	4A	Window	
6008	5008	4B	Middle	
6009	5009	5A	Window	
6010	5010	5B	Middle	
6011	5011	6A	Window	
6012	5012	6B	Middle	
6013	5013	7A	Window	
6014	5014	7B	Middle	
6015	5015	8A	Window	
6016	5016	8B	Middle	
6017	5017	9A	Window	
6018	5018	9B	Middle	
6019	5019	10A	Window	

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
SEAT_ID	BOOKING_ID	SEAT_NO	SEAT_TYPE	
6020	5020	10B	Middle	
6021	5021	11A	Window	
6022	5022	11B	Middle	
6023	5023	12A	Window	
6024	5024	12B	Middle	
6025	5025	13A	Window	
6026	5026	13B	Middle	
6027	5027	14A	Window	
6028	5028	14B	Middle	
6029	5029	15A	Window	
6030	5030	15B	Middle	
6031	5031	16A	Window	
6032	5032	16B	Middle	
6033	5033	17A	Window	
6034	5034	17B	Middle	
6035	5035	18A	Window	
6036	5036	18B	Middle	
6037	5037	19A	Window	
6038	5038	19B	Middle	
6039	5039	20A	Window	
6040	5040	20B	Middle	
NULL	NULL	NULL	NULL	

8. Insert Into Payment Table

INSERT INTO payment

(PAYMENT_ID, BOOKING_ID, PAYMENT_DATE, PAYMENT_METHOD,
PAYMENT_STATUS, AMOUNT)

VALUES

(7001,5001,'2026-08-01','UPI','Paid',5500.00),
(7002,5002,'2026-08-02','Card','Paid',9500.00),
(7003,5003,'2026-08-03','UPI','Paid',6200.00),
(7004,5004,'2026-08-04','Card','Paid',4800.00),
(7005,5005,'2026-08-04','Net Banking','Paid',8500.00),
(7006,5006,'2026-08-05','UPI','Paid',6700.00),
(7007,5007,'2026-08-05','Card','Refunded',0.00),
(7008,5008,'2026-08-05','UPI','Paid',4300.00),
(7009,5009,'2026-08-05','Card','Paid',9000.00),
(7010,5010,'2026-08-06','UPI','Paid',18500.00),
(7011,5011,'2026-08-06','Card','Paid',32000.00),
(7012,5012,'2026-08-06','UPI','Paid',22000.00),
(7013,5013,'2026-08-07','Card','Paid',22500.00),
(7014,5014,'2026-08-07','UPI','Paid',3200.00),
(7015,5015,'2026-08-07','Card','Paid',6500.00),
(7016,5016,'2026-08-08','UPI','Paid',4200.00),
(7017,5017,'2026-08-08','Card','Paid',5500.00),
(7018,5018,'2026-08-08','UPI','Paid',6200.00),
(7019,5019,'2026-08-09','Card','Paid',8000.00),
(7020,5020,'2026-08-09','UPI','Paid',5100.00),
(7021,5021,'2026-08-09','Card','Paid',6700.00),
(7022,5022,'2026-08-10','UPI','Paid',10500.00),
(7023,5023,'2026-08-10','Card','Paid',4300.00),
(7024,5024,'2026-08-10','UPI','Paid',5900.00),
(7025,5025,'2026-08-11','Card','Paid',28000.00),
(7026,5026,'2026-08-11','UPI','Paid',20500.00),
(7027,5027,'2026-08-11','Card','Paid',35000.00),
(7028,5028,'2026-08-12','UPI','Paid',22500.00),

(7029,5029,'2026-08-12','Card','Paid',3200.00),
 (7030,5030,'2026-08-12','UPI','Paid',6500.00),
 (7031,5031,'2026-08-13','Card','Paid',4200.00),
 (7032,5032,'2026-08-13','UPI','Paid',5500.00),
 (7033,5033,'2026-08-13','Card','Paid',9500.00),
 (7034,5034,'2026-08-14','UPI','Paid',4800.00),
 (7035,5035,'2026-08-14','Card','Paid',5100.00),
 (7036,5036,'2026-08-14','UPI','Paid',11500.00),
 (7037,5037,'2026-08-15','Card','Paid',6900.00),
 (7038,5038,'2026-08-15','UPI','Paid',4300.00),
 (7039,5039,'2026-08-15','Card','Paid',9000.00),
 (7040,5040,'2026-08-16','UPI','Paid',18500.00);

SELECT * from payment;

Result Grid						
Filter Rows:			Edit:		Export/Import:	
			Wrap Cell Contents:			
	PAYMENT_ID	BOOKING_ID	PAYMENT_DATE	PAYMENT_METHOD	PAYMENT_STATUS	AMOUNT
▶	7001	5001	2026-08-01	UPI	Paid	5500.00
	7002	5002	2026-08-02	Card	Paid	9500.00
	7003	5003	2026-08-03	UPI	Paid	6200.00
	7004	5004	2026-08-04	Card	Paid	4800.00
	7005	5005	2026-08-04	Net Banking	Paid	8500.00
	7006	5006	2026-08-05	UPI	Paid	6700.00
	7007	5007	2026-08-05	Card	Refunded	0.00
	7008	5008	2026-08-05	UPI	Paid	4300.00
	7009	5009	2026-08-05	Card	Paid	9000.00
	7010	5010	2026-08-06	UPI	Paid	18500.00
	7011	5011	2026-08-06	Card	Paid	32000.00
	7012	5012	2026-08-06	UPI	Paid	22000.00
	7013	5013	2026-08-07	Card	Paid	22500.00
	7014	5014	2026-08-07	UPI	Paid	3200.00
	7015	5015	2026-08-07	Card	Paid	6500.00
	7016	5016	2026-08-08	UPI	Paid	4200.00
	7017	5017	2026-08-08	Card	Paid	5500.00
	7018	5018	2026-08-08	UPI	Paid	6200.00
	7019	5019	2026-08-09	Card	Paid	8000.00

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	PAYMENT_ID	BOOKING_ID	PAYMENT_DATE	PAYMENT_METHOD	PAYMENT_STATUS	AMOUNT
	7020	5020	2026-08-09	UPI	Paid	5100.00
	7021	5021	2026-08-09	Card	Paid	6700.00
	7022	5022	2026-08-10	UPI	Paid	10500.00
	7023	5023	2026-08-10	Card	Paid	4300.00
	7024	5024	2026-08-10	UPI	Paid	5900.00
	7025	5025	2026-08-11	Card	Paid	28000.00
	7026	5026	2026-08-11	UPI	Paid	20500.00
	7027	5027	2026-08-11	Card	Paid	35000.00
	7028	5028	2026-08-12	UPI	Paid	22500.00
	7029	5029	2026-08-12	Card	Paid	3200.00
	7030	5030	2026-08-12	UPI	Paid	6500.00
	7031	5031	2026-08-13	Card	Paid	4200.00
	7032	5032	2026-08-13	UPI	Paid	5500.00
	7033	5033	2026-08-13	Card	Paid	9500.00
	7034	5034	2026-08-14	UPI	Paid	4800.00
	7035	5035	2026-08-14	Card	Paid	5100.00
	7036	5036	2026-08-14	UPI	Paid	11500.00
	7037	5037	2026-08-15	Card	Paid	6900.00
	7038	5038	2026-08-15	UPI	Paid	4300.00
	7039	5039	2026-08-15	Card	Paid	9000.00
	7040	5040	2026-08-16	UPI	Paid	18500.00
	NULL	NULL	NULL	NULL	NULL	NULL

Questions

Q.1 Find the total number of passengers in the passenger table.

COUNT()

```
SELECT COUNT(*) AS total_passengers
```

```
FROM passenger;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_passengers			
▶	40			

Q.2 Find the total revenue generated from all bookings using TOTAL_AMOUNT.

SUM()

```
SELECT SUM(TOTAL_AMOUNT) AS total_revenue
```

```
FROM booking;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_revenue			
▶	410600.00			

Q.3 Find the average BASE_FARE of all flights.

AVG()

```
SELECT AVG(BASE_FARE) AS average_fare
```

```
FROM flight;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	average_fare			
▶	9320.000000			

Q.4 Find the highest and lowest flight base fare.

MAX() + MIN()

SELECT

MAX(BASE_FARE) AS highest_fare,

MIN(BASE_FARE) AS lowest_fare

FROM flight;

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	highest_fare	lowest_fare			
▶	22500.00	3200.00			

Q.5 Display all passengers whose age is greater than 35.

WHERE

SELECT *

FROM passenger

WHERE AGE > 35;

Result Grid										Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
	PASSENGER_ID	FIRST_NAME	LAST_NAME	GENDER	AGE	PHONE_NO	EMAIL	CITY	PASSPORT_NO				
▶	5	Rohan	Mehta	Male	42	9876500005	rohan.mehta@gmail.com	Ahmedabad	P100005				
	7	Vikram	Singh	Male	38	9876500007	vikram.singh@gmail.com	Jaipur	P100007				
	10	Kavita	Shah	Female	40	9876500010	kavita.shah@gmail.com	Mumbai	P100010				
	11	Sanjay	Rao	Male	45	9876500011	sanjay.rao@gmail.com	Bengaluru	P100011				
	15	Nikhil	Gupta	Male	36	9876500015	nikhil.gupta@gmail.com	Pune	P100015				
	19	Manish	Yadav	Male	41	9876500019	manish.yadav@gmail.com	Lucknow	P100019				
	21	Varun	Bose	Male	37	9876500021	varun.bose@gmail.com	Kolkata	P100021				
	23	Deepak	Mishra	Male	48	9876500023	deepak.mishra@gmail.com	Varanasi	P100023				
	25	Mohit	Agarwal	Male	39	9876500025	mohit.agarwal@gmail.com	Jaipur	P100025				
	27	Suresh	Pillai	Male	52	9876500027	suresh.pillai@gmail.com	Kochi	P100027				
	33	Raj	Chavan	Male	44	9876500033	raj.chavan@gmail.com	Mumbai	P100033				
	34	Swati	Naik	Female	36	9876500034	swati.naik@gmail.com	Goa	P100034				
	37	Abhishek	Jain	Male	40	9876500037	abhishek.jain@gmail.com	Indore	P100037				
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL				

Q.6 Find the number of passengers from each city.

GROUP BY + COUNT

```
SELECT CITY, COUNT(*) AS passenger_count
```

```
FROM passenger
```

```
GROUP BY CITY;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
CITY	passenger_count		
Mumbai	7		
Pune	5		
Delhi	5		
Ahmedabad	1		
Jaipur	2		
Kochi	3		
Bengaluru	3		
Chennai	1		
Kolkata	4		
Amritsar	1		
Lucknow	2		
Varanasi	1		
Hyderabad	1		
Goa	1		
Indore	1		
Patna	1		
Nagpur	1		

Q.7 Find the total booking amount generated by each class.

GROUP BY + SUM()

```
SELECT
```

```
CLASS,
```

```
SUM(TOTAL_AMOUNT) AS total_revenue
```

```
FROM booking
```

```
GROUP BY CLASS;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
CLASS	total_revenue		
Economy	227100.00		
Business	183500.00		

Q.8 Display each flight number along with its airline name.

INNER JOIN

SELECT

f.FLIGHT_NO,

a.AIRLINE_NAME

FROM flight f

INNER JOIN airline a

ON f.AIRLINE_ID = a.AIRLINE_ID;

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	FLIGHT_NO	AIRLINE_NAME			
▶	AI101	Air India			
	AI102	Air India			
	AI205	Air India			
	AI410	Air India			
	AI510	Air India			
	6E201	IndiGo	Air India		
	6E202	IndiGo			
	6E305	IndiGo			
	6E411	IndiGo			
	UK301	Vistara			
	UK302	Vistara			
	EK501	Emirates			
	EK502	Emirates			
	SQ421	Singapore Airli...			
	SQ422	Singapore Airli...			

Q.9 Display all passengers and their booking IDs, including passengers who have not made any booking.

LEFT JOIN

SELECT

p.PASSENGER_ID,

p.FIRST_NAME,

p.LAST_NAME,

b.BOOKING_ID

FROM passenger p

LEFT JOIN booking b

ON p.PASSENGER_ID = b.PASSENGER_ID;

Result Grid					Filter Rows:	Export:	Wrap Cell Content:
PASSENGER_ID	FIRST_NAME	LAST_NAME	BOOKING_ID				
21	Varun	Bose	5021				
22	Tanya	Sen	5022				
23	Deepak	Mishra	5023				
24	Isha	Reddy	5024				
25	Mohit	Agarwal	5025				
26	Divya	Menon	5026				
27	Suresh	Pillai	5027				
28	Nandini	Rao	5028				
29	Akash	Patil	5029				
30	Shreya	Das	5030				
31	Harsh	Thakur	5031				
32	Mansi	Joshi	5032				
33	Raj	Chavan	5033				
34	Swati	Naik	5034				
35	Yash	Saxena	5035				
36	Komal	Bhat	5036				
37	Abhishek	Jain	5037				
38	Aarti	Sinha	5038				
39	Ritesh	Tiwari	5039				
40	Tanvi	Deshmukh	5040				

Result Grid  Filter Rows: Export:  Wrap Cell Content: 				
	PASSENGER_ID	FIRST_NAME	LAST_NAME	BOOKING_ID
	21	Varun	Bose	5021
	22	Tanya	Sen	5022
	23	Deepak	Mishra	5023
	24	Isha	Reddy	5024
	25	Mohit	Agarwal	5025
	26	Divya	Menon	5026
	27	Suresh	Pillai	5027
	28	Nandini	Rao	5028
	29	Akash	Patil	5029
	30	Shreya	Das	5030
	31	Harsh	Thakur	5031
	32	Mansi	Joshi	5032
	33	Raj	Chavan	5033
	34	Swati	Naik	5034
	35	Yash	Saxena	5035
	36	Komal	Bhat	5036
	37	Abhishek	Jain	5037
	38	Aarti	Sinha	5038
	39	Ritesh	Tiwari	5039
	40	Tanvi	Deshmukh	5040

Q.10 Display all bookings along with passenger names using a RIGHT JOIN.

RIGHT JOIN

SELECT

p.FIRST_NAME,

p.LAST_NAME,

b.BOOKING_ID,

b.TOTAL_AMOUNT

FROM passenger p

RIGHT JOIN booking b

ON p.PASSENGER_ID = b.PASSENGER_ID;

Result Grid		Filter Rows:	Export:	Wrap Cell Content: IA
	FIRST_NAME	LAST_NAME	BOOKING_ID	TOTAL_AMOUNT
	Varun	Bose	5021	6700.00
	Tanya	Sen	5022	10500.00
	Deepak	Mishra	5023	4300.00
	Isha	Reddy	5024	5900.00
	Mohit	Agarwal	5025	28000.00
	Divya	Menon	5026	20500.00
	Suresh	Pillai	5027	35000.00
	Nandini	Rao	5028	22500.00
	Akash	Patil	5029	3200.00
	Shreya	Das	5030	6500.00
	Harsh	Thakur	5031	4200.00
	Mansi	Joshi	5032	5500.00
	Raj	Chavan	5033	9500.00
	Swati	Naik	5034	4800.00
	Yash	Saxena	5035	5100.00
	Komal	Bhat	5036	11500.00
	Abhishek	Jain	5037	6900.00
	Aarti	Sinha	5038	4300.00
	Ritesh	Tiwari	5039	9000.00
	Tanvi	Deshmukh	5040	18500.00

Result Grid		Filter Rows:	Export:	Wrap Cell Content: IA
	FIRST_NAME	LAST_NAME	BOOKING_ID	TOTAL_AMOUNT
►	Rahul	Sharma	5001	5500.00
	Priya	Patel	5002	9500.00
	Amit	Verma	5003	6200.00
	Sneha	Desai	5004	4800.00
	Rohan	Mehta	5005	8500.00
	Anjali	Kumar	5006	6700.00
	Vikram	Singh	5007	0.00
	Neha	Joshi	5008	4300.00
	Arjun	Nair	5009	9000.00
	Kavita	Shah	5010	18500.00
	Sanjay	Rao	5011	32000.00
	Pooja	Iyer	5012	22000.00
	Karan	Malhotra	5013	22500.00
	Riya	Kapoor	5014	3200.00
	Nikhil	Gupta	5015	6500.00
	Meera	Chopra	5016	4200.00
	Aditya	Kulkarni	5017	5500.00
	Simran	Kaur	5018	6200.00
	Manish	Yadav	5019	8000.00
	Ayesha	Khan	5020	5100.00
	Varun	Bose	5021	6700.00

Q.11 Find the total booking revenue generated by each airline.

INNER JOIN + SUM() + GROUP BY

SELECT

a.AIRLINE_NAME,

SUM(b.TOTAL_AMOUNT) AS total_revenue

FROM airline a

INNER JOIN flight f

ON a.AIRLINE_ID = f.AIRLINE_ID

INNER JOIN booking b

ON f.FLIGHT_ID = b.FLIGHT_ID

GROUP BY a.AIRLINE_NAME;

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	AIRLINE_NAME	total_revenue			
▶	Air India	75600.00			
	IndiGo	73200.00			
	Vistara	42300.00			
	Emirates	117500.00			
	Singapore Airlines	102000.00			

Q.12 Find the number of bookings made for each airline.

INNER JOIN + COUNT() + GROUP BY

SELECT

a.AIRLINE_NAME,

COUNT(b.BOOKING_ID) AS booking_count

FROM airline a

INNER JOIN flight f

ON a.AIRLINE_ID = f.AIRLINE_ID

INNER JOIN booking b

ON f.FLIGHT_ID = b.FLIGHT_ID

GROUP BY a.AIRLINE_NAME;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	AIRLINE_NAME	booking_count
▶	Air India	14
	IndiGo	11
	Vistara	6
	Emirates	5
	Singapore Airlines	4

Q.13 Find airlines having more than 5 bookings.

GROUP BY + HAVING

SELECT

a.AIRLINE_NAME,

COUNT(b.BOOKING_ID) AS booking_count

FROM airline a

INNER JOIN flight f

ON a.AIRLINE_ID = f.AIRLINE_ID

INNER JOIN booking b

ON f.FLIGHT_ID = b.FLIGHT_ID

GROUP BY a.AIRLINE_NAME

HAVING COUNT(b.BOOKING_ID) > 5;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	AIRLINE_NAME	booking_count
▶	Air India	14
	IndiGo	11
	Vistara	6

Q.14 Display the flight number, airline name, and base fare for flights whose base fare is greater than ₹10,000.

JOIN + WHERE

Result Grid		 Filter Rows:	Export: 	Wrap Cell Content: 
	FLIGHT_NO	AIRLINE_NAME	BASE_FARE	
▶	EK501	Emirates	18500.00	
	EK502	Emirates	20500.00	
	SQ421	Singapore Airlines	22000.00	
	SQ422	Singapore Airlines	22500.00	

WHERE + AND

```
SELECT *  
FROM passenger  
WHERE GENDER = 'Female'  
AND AGE > 30;
```

[illegible]

Q.16 Find bookings whose TOTAL_AMOUNT is greater than the average booking amount.

Subquery + AVG()

```
SELECT *  
FROM booking  
WHERE TOTAL_AMOUNT > (  
SELECT AVG(TOTAL_AMOUNT)  
FROM booking  
);
```

Result Grid							
		Filter Rows:		Edit:		Export/Import:	
						Wrap Cell Content:	
	BOOKING_ID	PASSENGER_ID	FLIGHT_ID	BOOKING_DATE	CLASS	BOOKING_STATUS	TOTAL_AMOUNT
▶	5010	10	309	2026-08-06	Economy	Confirmed	18500.00
	5011	11	310	2026-08-06	Business	Confirmed	32000.00
	5012	12	311	2026-08-06	Economy	Confirmed	22000.00
	5013	13	312	2026-08-07	Economy	Confirmed	22500.00
	5022	22	306	2026-08-10	Business	Confirmed	10500.00
	5025	25	309	2026-08-11	Business	Confirmed	28000.00
	5026	26	310	2026-08-11	Economy	Confirmed	20500.00
	5027	27	311	2026-08-11	Business	Confirmed	35000.00
	5028	28	312	2026-08-12	Economy	Confirmed	22500.00
	5036	36	305	2026-08-14	Business	Confirmed	11500.00
	5040	40	309	2026-08-16	Economy	Confirmed	18500.00
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q.17 Display booking ID and categorize the booking amount as Low, Medium, or High.

CASE

```
SELECT  
BOOKING_ID,  
TOTAL_AMOUNT,  
CASE  
WHEN TOTAL_AMOUNT < 10000 THEN 'Low'  
WHEN TOTAL_AMOUNT BETWEEN 10000 AND 20000 THEN 'Medium'  
ELSE 'High'
```

END AS amount_category

FROM booking;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
BOOKING_ID	TOTAL_AMOUNT	amount_category	
5001	5500.00	Low	
5002	9500.00	Low	
5003	6200.00	Low	
5004	4800.00	Low	
5005	8500.00	Low	
5006	6700.00	Low	
5007	0.00	Low	
5008	4300.00	Low	
5009	9000.00	Low	
5010	18500.00	Medium	
5011	32000.00	High	
5012	22000.00	High	
5013	22500.00	High	
5014	3200.00	Low	
5015	6500.00	Low	
5016	4200.00	Low	
5017	5500.00	Low	
5018	6200.00	Low	
5019	8000.00	Low	
5020	5100.00	Low	

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
BOOKING_ID	TOTAL_AMOUNT	amount_category	
5021	6700.00	Low	
5022	10500.00	Medium	
5023	4300.00	Low	
5024	5900.00	Low	
5025	28000.00	High	
5026	20500.00	High	
5027	35000.00	High	
5028	22500.00	High	
5029	3200.00	Low	
5030	6500.00	Low	
5031	4200.00	Low	
5032	5500.00	Low	
5033	9500.00	Low	
5034	4800.00	Low	
5035	5100.00	Low	
5036	11500.00	Medium	
5037	6900.00	Low	
5038	4300.00	Low	
5039	9000.00	Low	
5040	18500.00	Medium	

Q.18 Using a CTE, calculate the total revenue generated by each airline.

CTE + SUM()

```
WITH airline_revenue AS (  
    SELECT  
        a.AIRLINE_NAME,  
        SUM(b.TOTAL_AMOUNT) AS total_revenue  
    FROM airline a  
    INNER JOIN flight f  
    ON a.AIRLINE_ID = f.AIRLINE_ID  
    INNER JOIN booking b  
    ON f.FLIGHT_ID = b.FLIGHT_ID  
    GROUP BY a.AIRLINE_NAME  
)  
SELECT *  
FROM airline_revenue;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	AIRLINE_NAME	total_revenue			
▶	Air India	75600.00			
	IndiGo	73200.00			
	Vistara	42300.00			
	Emirates	117500.00			
	Singapore Airlines	102000.00			

Q.19 Rank passengers according to their total booking expenditure.

RANK() + Window Function

```
SELECT  
    p.PASSENGER_ID,  
    p.FIRST_NAME,  
    p.LAST_NAME,  
    SUM(b.TOTAL_AMOUNT) AS total_spent,  
    RANK() OVER (
```

ORDER BY SUM(b.TOTAL_AMOUNT) DESC

) AS spending_rank

FROM passenger p

INNER JOIN booking b

ON p.PASSENGER_ID = b.PASSENGER_ID

GROUP BY

p.PASSENGER_ID,

p.FIRST_NAME,

p.LAST_NAME;

Result Grid						Filter Rows:	Export:	Wrap Cell Content:
	PASSENGER_ID	FIRST_NAME	LAST_NAME	total_spent	spending_rank			
▶	27	Suresh	Pillai	35000.00	1			
	11	Sanjay	Rao	32000.00	2			
	25	Mohit	Agarwal	28000.00	3			
	13	Karan	Malhotra	22500.00	4			
	28	Nandini	Rao	22500.00	4			
	12	Pooja	Iyer	22000.00	6			
	26	Divya	Menon	20500.00	7			
	40	Tanvi	Deshmukh	18500.00	8			
	10	Kavita	Shah	18500.00	8			
	36	Komal	Bhat	11500.00	10			
	22	Tanya	Sen	10500.00	11			
	2	Priya	Patel	9500.00	12			
	33	Raj	Chavan	9500.00	12			
	39	Ritesh	Tiwari	9000.00	14			
	9	Arjun	Nair	9000.00	14			
	5	Rohan	Mehta	8500.00	16			
	19	Manish	Yadav	8000.00	17			
	37	Abhishek	Jain	6900.00	18			
	21	Varun	Bose	6700.00	19			
	6	Anjali	Kumar	6700.00	19			

Result Grid Filter Rows: Export: Wrap Cell Content:					
	PASSENGER_ID	FIRST_NAME	LAST_NAME	total_spent	spending_rank
	15	Nikhil	Gupta	6500.00	21
	30	Shreya	Das	6500.00	21
	3	Amit	Verma	6200.00	23
	18	Simran	Kaur	6200.00	23
	24	Isha	Reddy	5900.00	25
	17	Aditya	Kulkarni	5500.00	26
	32	Mansi	Joshi	5500.00	26
	1	Rahul	Sharma	5500.00	26
	20	Ayesha	Khan	5100.00	29
	35	Yash	Saxena	5100.00	29
	34	Swati	Naik	4800.00	31
	4	Sneha	Desai	4800.00	31
	23	Deepak	Mishra	4300.00	33
	8	Neha	Joshi	4300.00	33
	38	Aarti	Sinha	4300.00	33
	16	Meera	Chopra	4200.00	36
	31	Harsh	Thakur	4200.00	36
	29	Akash	Patil	3200.00	38
	14	Riya	Kapoor	3200.00	38
	7	Vikram	Singh	0.00	40

CONCLUSION

The Airline Reservation System project successfully demonstrates how SQL can be used to design and manage a relational database for airline operations. The database contains interconnected tables such as Airline, Airport, Aircraft, Flight, Passenger, Booking, Seat, and Payment, with primary and foreign keys used to maintain relationships and data integrity.

Through this project, various SQL concepts were implemented, including data retrieval using SELECT and WHERE, sorting, INNER JOINS, aggregate functions, GROUP BY, HAVING, subqueries, CASE statements, CTEs, and window functions. These operations make it possible to analyze passenger information, flight details, bookings, revenues, payments, and airline performance.

Overall, the project provides practical experience in database design, SQL querying, data management, and analysis. It shows how a well-structured relational database can organize airline reservation data and help generate meaningful information for managing flights, passengers, bookings, seats, and payments.